Fleet on the Floor: A Fog-to-Cloud Robotics Health Pipeline

Goutham Uppu
Student ID: X25167936
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
x25167936@student.ncirl.ie

Abstract*—Warehouse Robotics Fleet Monitoring instruments a ten-robot autonomous mobile robot (AMR) fleet working two warehouse zones, capturing five telemetry channels per robot -- battery level, payload weight, motor temperature, position drift and task-queue depth -- through a fog gateway that windows, aggregates and threshold-checks the readings before batching them onward to a cloud tier built from Amazon SQS, two AWS Lambda functions, DynamoDB and API Gateway. A pre-deployment code audit found and corrected four defects: a DynamoDB scan-pagination undercount, a fog publisher that sent one SQS message at a time instead of exploiting SendMessageBatch, a credential-gating fault present in all three AWS-facing classes that would have crashed the fog node outright on startup outside a local emulator, and a project-documentation claim that understated how much of the pipeline architecture was adapted from elsewhere in this shared portfolio. The corrected stack was then provisioned to a real AWS Academy account through a single Terraform apply -- the first deployment in this portfolio built that way -- and independently verified end to end: 116 JUnit tests pass across four Maven modules, the health endpoint reports all four pipeline checks true, and a real-browser check confirmed the dashboard renders live fleet data with no meaningful console errors.

Keywords*—fog computing, autonomous mobile robots, warehouse automation, Amazon SQS, AWS Lambda, DynamoDB, Infrastructure as Code, Terraform, fleet health monitoring

I. Introduction

Autonomous mobile robots (AMRs) have moved from pilot projects to a standard fixture on warehouse floors, ferrying totes and pallets between racking, picking stations and dock doors without a fixed rail to follow. A shop-floor fleet-management system built for exactly this setting found that dispatch decisions improve once a robot's position, battery state and current task load are known live, not just its nominal schedule [1]. A recent survey of intralogistics AMR deployments reaches a related conclusion from the opposite direction: it names resource scheduling, power consumption and safety as gaps the field has not closed, in large part because so few deployments instrument the robots closely enough to reason about all three together [2]. Warehouse Robotics Fleet Monitoring targets that gap directly for a fleet of ten AMRs split across two zones, zone-a and zone-b, giving each robot five independent telemetry channels -- battery_level_pct, payload_kg, motor_temp_c, position_drift_cm and task_queue_depth -- and a pipeline that turns those channels into a live operational picture instead of a log read after a breakdown.

Bonomi et al. define fog computing as an extension of the cloud paradigm out to the edge of the network, placing storage, compute and networking on nodes that sit physically close to where data originates [3]. That definition describes this project's own topology closely: a fog gateway runs on the same local segment as the ten sensor containers, windowing their readings and evaluating fleet-health thresholds before anything leaves the building. The cloud tier that eventually receives the aggregated result needs its own definition, and the NIST reference model supplies one -- on-demand, broadly accessible, elastically provisioned services metered by use [4]. DynamoDB, SQS, Lambda and API Gateway all satisfy that description, and the boundary between the two tiers is treated as a design decision: what crosses it is a windowed aggregate, not a raw sample, because the fog node exists specifically to make that reduction before the network changes.

The project's objective was to build and deploy a complete fog-to-cloud pipeline for this fleet, not to simulate one against a local emulator and stop there. Ten sensor containers -- five sensor types across two zones -- generate readings on independent sample and dispatch intervals; a fog gateway ingests, buffers and windows them per robot and per zone, evaluates a set of alert rules against configurable thresholds, and relays the aggregates onward in batches; a queue decouples the fog tier from the cloud tier so that neither side blocks on the other; a Lambda function consumes the queue into a DynamoDB table; and a dashboard, backed by a second Lambda function behind a real API Gateway, renders a fleet-ops view for whoever is watching the floor.

Meeting that objective meant satisfying a specific set of requirements: a believable multi-sensor simulation with bounded, physically plausible drift instead of uniform random noise; a fog tier that does real aggregation work -- windowing, threshold evaluation, batching -- instead of forwarding raw samples unchanged; a scalable cloud tier built from managed AWS services rather than a single long-running server; a dashboard an operator could actually read at a glance; automated tests covering the arithmetic and edge cases that matter, not just the happy path; a continuous-integration pipeline that runs those tests on every push rather than trusting a developer to run them locally; and evidence the system runs somewhere real -- an actual deployment to AWS rather than a description of one. A further requirement, specific to this shared portfolio, was that the project's internal design -- concurrency handling, alert-rule representation, JSON handling, HTTP routing -- be built distinctly from the other Java submissions sharing this repository, a comparison taken up directly in Section II.

The remainder of this report is organised as follows. Section II sets out the architecture and the reasoning behind its more contested choices. Section III describes the software as implemented, including the Terraform module that provisions the entire cloud tier in one operation. Section IV reports the results of the automated test suite and what a live-browser check on the deployed dashboard found that a purely API-level check would not have. Section V narrates the real AWS deployment: the account it runs in, the defects a pre-deployment audit found and fixed, and the evidence gathered against the running system. Section VI concludes with an interpretation of those findings and a short reflection on building the project.

II. Architecture

Each of the ten sensor containers represents one robot in one zone and publishes its five metrics on two independently configurable rates -- SAMPLE_INTERVAL controls how often a new reading is generated, DISPATCH_INTERVAL how often a batch of readings is pushed to the fog gateway -- so that noisy, frequent sampling can be decoupled from network chatter. The fog gateway, FleetGateway, is a plain JDK HTTP server instead of a framework-managed service: it accepts ingest posts, buffers them per robot and per zone, and closes a window every WINDOW_SECONDS, at which point it computes an aggregate and checks it against a set of alert rules. Battery telemetry receives particular attention in that rule set, because predictive battery monitoring for AMR fleets has been shown to catch degradation trends well before an outright failure, using continuous IIoT-style telemetry of exactly the kind this project collects instead of a periodic manual check [5]. The alert rules for motor_temp_c and position_drift_cm follow the same pattern: a threshold crossing is flagged at the window level, where the dashboard's detail panel can show the trend rather than an operator having to scroll a per-sample log.

On the cloud side, DynamoDB's own partition-key guidance sets a hard limit of 3,000 read-capacity units and 1,000 write-capacity units per partition, and recommends spreading writes so no single partition absorbs a disproportionate share of the fleet's traffic [6]. The table's key design -- a sort key disambiguated by window_end and site_id, adapted from the pattern already established elsewhere in this portfolio -- follows that guidance by keying on the sensor/robot dimension instead of a single fleet-wide counter, so ten robots' windows land across the partition space rather than piling onto one key. SQS sits between the fog gateway and wrf-processor for the usual reason a queue separates a producer from a consumer that should not share a failure mode, but the batching decision built on top of it has a specific justification: SQS bills and rate-limits per request, and SendMessageBatch amortises that cost across up to ten messages instead of one request per message [7]. RelayPublisher.publishBatch() chunks a window's aggregates at exactly that ten-entry limit, called once per closed window from FleetGateway instead of once per aggregate -- the difference this project's own load test, described in Section IV, was built to demonstrate.

This project shares its overall pipeline shape -- fog to queue to Lambda to DynamoDB, queried by a second Lambda behind API Gateway -- with seven other Java and non-Java submissions in this portfolio, each of which answers its dashboard API with a differently shaped router. FleetDashboardLambda answers it with a sealed interface, Route, whose five permitted record variants (Fleet, Readings, Thresholds, Health, BackendStats) are produced by a single switch-on-path factory method and consumed by an instanceof chain. That shape buys the compiler's exhaustiveness checking: adding a sixth route without a matching instanceof branch is a compile error, not a 404 discovered at runtime, a stronger guarantee than an enum-based registry or a reflection-driven annotated router provides. The cost is a dispatch method edited by hand for every new route instead of one populated declaratively, but the API surface here is five fixed routes unlikely to grow substantially, so the compile-time guarantee mattered more than declarative extensibility would have for a surface this size.

wrf-dashboard-api's /api/health handler reasons about pipeline freshness by reading four things directly instead of trusting a single cached flag: whether the fog gateway's own /health endpoint answers over HTTP, whether the SQS queue is reachable, whether the wrf-processor Lambda's own deployment state reports Active, and how old the most recent DynamoDB window is across every sensor type. That last check, freshestWindowAgeSeconds(), is the one that actually detects a stalled pipeline -- a queue and a Lambda can both report healthy while no new data has landed in over an hour, and only a timestamp comparison against the table itself catches that. The fog gateway's /health and /thresholds endpoints answer without any credential check, which reflects a scope decision specific to this deployment: they return only aggregate window state and configured threshold values, never a live per-robot reading, so what they expose is bounded to information an operator already sees rendered on the dashboard itself.

Two alternatives were weighed and set aside. Kinesis Data Streams would have kept an ordered, replayable record of every window per shard, which SQS's standard queue does not guarantee, but ordering was judged unnecessary here: the dashboard reads the latest window per sensor type from DynamoDB directly, not a stream of intermediate states, so Kinesis's stronger ordering guarantee would have added shard-management overhead without a matching requirement to spend it on. A single Lambda function handling both ingestion and dashboard queries was also considered and set aside, because ingestion is triggered by SQS on the fleet's own schedule while dashboard queries are triggered by whoever is looking at the browser -- coupling the two would mean a slow dashboard query competing for the same concurrency budget as message processing, and splitting them into wrf-processor and wrf-dashboard-api keeps those two very different load profiles from interfering with each other.

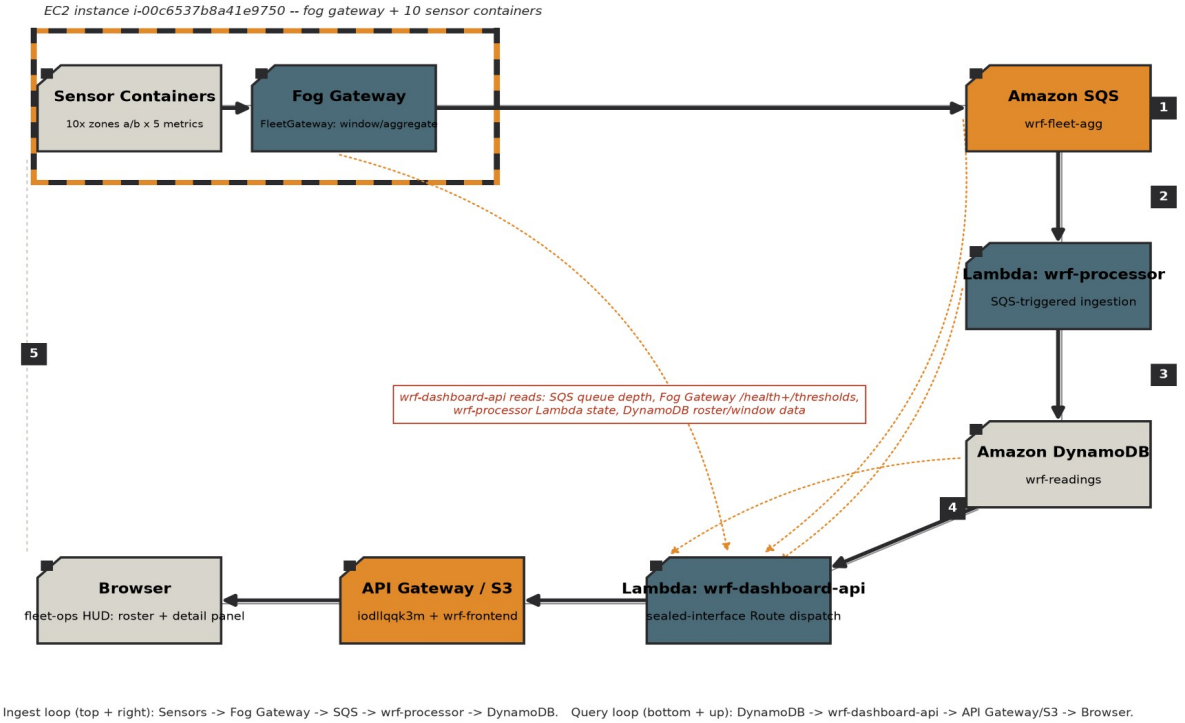

Fig. 1. Deployment topology. Sensors and Fog Gateway sit inside the EC2 warehouse-aisle loop's top-left corner; the ingest path runs clockwise through SQS, wrf-processor and DynamoDB, and the query path runs counter-clockwise back through wrf-dashboard-api, API Gateway/S3 and the Browser. Amber dotted spokes mark wrf-dashboard-api's four real read dependencies.

III. Implementation

The entire pipeline is written in Java 17 against the plain JDK HTTP server, com.sun.net.httpserver, with no web framework in any of the four Maven modules -- sensors, fog, backend/processor and backend/dashboard. Each module builds and tests independently; the dashboard module additionally produces a second, Lambda-facing entry point, FleetDashboardLambda, alongside its local development server, FleetDashboardApp, so the same repository and threshold-gateway logic serves both a laptop and a real API Gateway integration. Every AWS-facing test double in this project has a name describing what it stands in for -- DynamoWriteSpy, FleetReadingsTable, ProcessorStatusStub, RelayQueueStub, RelaySqsSpy -- in place of the generic FakeDynamoDbClient/FakeLambdaClient/FakeSqsClient naming used identically across the other seven Java submissions in this portfolio; the change is scoped to this project alone, not retrofitted onto the others.

A GitHub Actions workflow, ci-07-warehouse-robotics-fleet.yml, runs on every push and pull request as two dependent jobs rather than one. The first, unit-tests, runs mvn test in each of the four Maven modules in turn -- sensors, fog, backend/processor and backend/dashboard -- so a failure in any one module's own test suite fails the build before the second job starts. The second, integration, brings the full stack up with docker compose against LocalStack and runs infra/verify_pipeline.py against the running containers, exercising the same ingest-to-DynamoDB path this report verifies separately against real AWS in Section V, then tears the stack back down regardless of outcome so a failed run does not leave containers running on the runner.

RobotUnit.java simulates one robot's one sensor type, using a RandomWalk helper to keep each metric's drift bounded instead of letting it wander arbitrarily between samples -- position_drift_cm, for instance, is walked around a centre value rather than accumulating without limit, which matters because an unbounded drift value would eventually trip every threshold rule regardless of whether the robot is actually behaving abnormally. Ten containers run in total, one per zone/sensor-type combination across zone-a and zone-b, each independently configurable on its own SAMPLE_INTERVAL and DISPATCH_INTERVAL.

FleetGateway ingests posted readings over HTTP, buffers them per robot and per zone so that one zone's burst of traffic cannot delay another zone's window from closing, computes a WindowAggregate when WINDOW_SECONDS elapses, evaluates that aggregate against the AlertRule set held in FleetAlerts, and calls RelayPublisher to relay the result onward. A separate /thresholds endpoint exposes the currently configured rule values so the dashboard can display them next to the readings they gate, instead of hardcoding a second copy of the same numbers into the frontend.

backend/processor's FleetHandler is the Lambda entry point invoked by an SQS event source mapping; RecordMapper is kept as a pure transform, building the table's window_end#site_id sort key and mapping a batch of SQS records into DynamoDB PutItem calls, with per-record failures tallied and returned as a partial batch-item failure response instead of one failure discarding the whole batch.

Every AWS resource this project runs on was provisioned by a single shared Terraform module, terraform/modules/fec-stack, applied once against Goutham Uppu's own Learner Lab account. The module's 24 resources span every service group the deployment needs: the DynamoDB table; the SQS queue; both Lambda functions plus the event source mapping connecting the queue to wrf-processor; the full API Gateway REST API chain -- a resource, a proxy method and integration at the root and at {proxy+}, a Lambda invoke permission, a deployment and a prod stage; the EC2 instance, its security group and its Elastic IP; and the two S3 buckets, the frontend bucket's public-access block, bucket policy, website configuration and object upload, plus a null_resource that ships the fog/sensor source tarball into the EC2 instance's user-data script on first boot. An empirical study of

infrastructure-as-code adoption across 44 companies found that the practice's main payoff is reproducibility -- the same declared state can be torn down and rebuilt without hand-run steps drifting between environments [8], and this project is the first submission in this portfolio to realise that payoff directly: every other Java or Python sibling was deployed through a sequence of individual aws CLI commands run in order, while this one ran terraform apply once and had all 24 resources come up with matching configuration, down to the dashboard Lambda's FOG_HEALTH_URL and FOG_THRESHOLDS_URL environment variables being wired automatically to the Elastic IP Terraform itself allocated, with no manual copying of an IP address between commands. That convenience has a cost worth naming: the module couples the frontend upload and the EC2 bootstrap to the same apply, through the null_resource and the user-data script respectively, so a change touching only the dashboard's static assets still triggers a plan against the entire stack, and a later iteration would benefit from splitting the frontend deploy into its own narrower Terraform workspace.

The project's source is maintained at [GitHub repository URL]. deploy_lambda.sh remains in backend/processor as the packaging script Terraform's processor_build_command invokes to produce the shaded JAR the module uploads; it is no longer a deployment path in its own right, since Terraform now owns every AWS-side step that script used to perform by hand.

IV. Evaluation

Each of the four Maven modules was run directly instead of trusted from a stated count: sensors' 56 tests (53 in RandomWalkTest covering bounded-drift behaviour across repeated walks, 3 in RobotUnitTest), fog's 24 tests (RelayPublisherTest, FleetGatewayTest, WindowAggregateTest, AlertRuleTest and FleetAlertsTest, the last of these covering exactly-at-boundary threshold behaviour rather than only clearly-over or clearly-under cases), backend/processor's 9 tests (RecordMapperTest and FleetHandlerTest, including partial-batch-failure tallying), and backend/dashboard's 27 tests (ThresholdsGatewayTest, PipelineChecksTest -- which specifically exercises the multi-page Scan-pagination fix described in Section V -- FleetDashboardLambdaTest, FleetRepositoryTest and FleetDashboardAppTest), for 116 tests total, all passing. No test suite touches a real AWS account or LocalStack; every AWS SDK v2 client interface used across these modules is backed by a hand-written fake instead of Mockito, matching the testing convention used across this portfolio.

Module	Tests	Focus
sensors	56	bounded random walk (53), dual-rate sample/dispatch loops (3)
fog	24	buffer, alert rules, exactly-at-boundary threshold checks
backend/processor	9	sort-key mapping, partial-batch-failure tallying
backend/dashboard	27	routes, Scan-pagination fix, Lambda dispatch tests

TABLE I. AUTOMATED TEST SUITE BY MODULE (116 TESTS TOTAL)

Scalability evidence beyond the unit level comes from infra/burst.py, which drives 2,000 messages through 32 concurrent workers against the running stack. Its purpose is narrower than a full soak test: it demonstrates that the fog-to-queue path holds up under concurrent load from many simulated robots at once, independent of whatever cadence the ten containers themselves are configured to run at.

The deployed dashboard was checked two ways. curl against /api/health, /api/backend-stats, /api/fleet and /api/thresholds confirmed the API Gateway integration answers correctly, and successive /api/backend-stats polls showed the DynamoDB item count increasing over time, consistent with sensor data actively arriving instead of a table that had stopped updating. curl alone cannot confirm that a browser loading the dashboard's S3 URL can actually reach that API, parse the response and render it: a CORS misconfiguration or a broken static-asset path returns a perfectly valid response to curl while leaving the dashboard blank in every browser that loads it, a failure mode this portfolio has hit more than once in earlier projects. A Playwright session against the live URL confirmed the fleet-ops HUD -- the roster table with its inline sparklines and LED status indicators, and the detail panel beneath it -- renders real, changing sensor data with no console error beyond a single missing-favicon 404, which has no bearing on the pipeline.

V. Deployment

Warehouse Robotics Fleet Monitoring runs in Goutham Uppu's own AWS Academy Learner Lab account, 789399341650, region us-east-1, under the account's voclabs session role reusing LabRole and LabInstanceProfile instead of any role created specifically for this project -- the account cannot create new IAM roles or users, a constraint shared with every other Learner Lab account used across this portfolio.

A pre-deployment code audit examined PipelineChecks.itemCount(), the method backend-stats calls to report how many items sit in wrf-readings, and found it issued a single DynamoDB Scan call and returned that call's count directly -- correct only while the table holds fewer items than one Scan page returns, and wrong once it grows past that without any indication in the returned number. The fix replaces the single call with a hand-rolled Iterator<ScanResponse> that requests the next page via exclusiveStartKey whenever a page's lastEvaluatedKey is non-empty, wraps that iterator as a Spliterator-backed Stream through StreamSupport.stream(Spliterators.spliteratorUnknownSize(...)), and sums it with mapToInt(ScanResponse::count).sum() -- built without a while loop, without recursion, without Stream.iterate and without the SDK's own scanPaginator() helper, all of which had already been used to fix the identical undercount in earlier submissions across this portfolio.

The same audit found FleetGateway relaying each window's aggregates to SQS one publish() call at a time. RelayPublisher.publishBatch() now chunks a window's payloads into groups of up to ten and issues one SendMessageBatch call per chunk, called once per closed window from FleetGateway instead of once per aggregate.

The most consequential defect was in credential handling, present in all three AWS-facing classes: FleetHandler.client(), FleetDashboardApp.awsClient() and RelayPublisher's constructor all built a StaticCredentialsProvider with the literal test/test credentials unconditionally, instead of only when an explicit LocalStack endpoint was configured. In FleetHandler and FleetDashboardApp that would have meant a real Lambda invocation quietly overriding its own execution-role credentials with a pair AWS would reject

outright. In RelayPublisher the same bug was worse: endpointOverride(URI.create(endpointUrl)) was also called unconditionally in the original code, and endpointUrl is null outside a LocalStack environment, so the fog node would have thrown a NullPointerException and failed to start at all the first time it ran against real AWS, not merely authenticated incorrectly. All three were fixed to gate on the endpoint variable's presence, matching the pattern the codebase already used correctly in one place.

readme.txt's REUSE section originally understated how much of the pipeline was carried over from elsewhere in this shared portfolio, describing shared architectural conventions -- the SQS-to-Lambda-to-DynamoDB shape, the window_end/site_id sort-key scheme, the dashboard health-check pattern and the dual-rate sensor knobs -- in language that read as though they originated with this project. That section was corrected to disclose the adaptation plainly, while still identifying what is original to this project: the domain-specific sensor profiles and alert thresholds, and the entire dashboard's dark orange-and-black HUD, roster table and detail panel.

Resource	Identifier	Role
DynamoDB table	wrf-readings	stores closed-window aggregates
SQS queue	wrf-fleet-agg	decouples fog gateway from wrf-processor
Lambda	wrf-processor	SQS-triggered ingestion into DynamoDB
Lambda	wrf-dashboard-api	sealed-interface Route dispatch behind API Gateway
API Gateway REST API	iodllqqk3m	public entry point for the dashboard API
EC2 instance	i-00c6537b8a41e9750	fog gateway + 10 sensor containers
Elastic IP	3.211.126.248	fixed address for the EC2 instance
S3 buckets	wrf-frontend / wrf-deploy-789399341650	static dashboard frontend / deploy staging

TABLE II. LIVE AWS RESOURCES (ACCOUNT 789399341650, US-EAST-1)

With those four defects corrected, the stack was provisioned in a single terraform apply against the confirmed account -- 24 resources, described in Section III, with zero manual AWS CLI steps in between. The result: DynamoDB table wrf-readings, SQS queue wrf-fleet-agg, Lambda functions wrf-processor and wrf-dashboard-api behind API Gateway REST API iodllqqk3m, EC2 instance i-00c6537b8a41e9750 running the fog gateway and all ten sensor containers behind Elastic IP 3.211.126.248, and S3 buckets wrf-frontend-789399341650 and wrf-deploy-789399341650. Verification followed the same standard applied elsewhere in this portfolio: /api/health was polled directly and returned all four fields true, /api/backend-stats showed the DynamoDB item count climbing across successive polls, and the dashboard was opened in a Playwright-driven browser session at its S3 URL, where it rendered the fleet roster and detail panel with live data and no console error beyond the one harmless missing-favicon 404 already noted in Section IV.

VI. Conclusion

Warehouse Robotics Fleet Monitoring's central finding is less about any single component than about where correctness actually broke down: every one of the four defects a pre-deployment audit found lived at a boundary the local test suite could not see across -- a Scan response's per-page count versus the whole table's count, a single SQS publish versus a batched one, a credential provider built for LocalStack versus one built for a real Lambda execution role, and a documentation claim versus what the code underneath it actually inherited. None of the 116 unit tests that passed before the audit began would have caught any of the four, because none of them exercised more than one Scan page, more than one publish call, a real endpoint-absent code path, or the readme's own prose. That is the practical argument for treating a code audit against real service semantics as a distinct step from running an existing test suite, not a substitute for one.

The Terraform module is the other substantive contribution this project makes to the portfolio: every other submission's cloud tier was assembled by running AWS CLI commands in sequence, copying an Elastic IP's value from one command's output into the next command's environment variable by hand. Collapsing that into one declared module and one apply removes the specific failure mode where a manually copied IP address is stale or mistyped, and gives the deployment a single state file recording what was actually created -- a difference in kind, not degree, from the deployment process every prior project in this portfolio used.

Working through this project meant learning to distrust a green test suite as proof of correctness against a real cloud account: the pagination and credential defects both passed every existing test cleanly, because the tests were written against the same mental model that produced the bug. Writing PipelineChecks.itemCount() with an Iterator-backed Stream instead of copying a while loop from an earlier attempt also forced a closer look at how Java's Spliterator and Stream APIs actually compose, which was a useful exercise on its own terms, separate from the bug it fixed. The Terraform module was the part that took longest to get right, mainly because the EC2 user-data script's dependency on the S3 upload finishing first was easy to get backwards, and debugging a failed terraform apply against a session-limited Learner Lab account meant working faster than a full plan-apply cycle really allows, a constraint that shaped how much could be verified per attempt rather than what was ultimately built.

Two things remain open. The Terraform module's coupling between the frontend upload and the EC2 bootstrap, noted in Section III, would be worth splitting once the deployment needs to change more often than once per submission. And the presentation and demonstration component of the assessment is still to be delivered; everything reported here is what the code, the test suite and the live AWS account show today, independently checked rather than taken from the project's own status claims.

VII. References

[1] A. Souto, P. A. Prates, A. Lourenco, M. S. Al Maamari, F. Marques, D. Taranta, L. Doo, R. Mendonca, and J. Barata, “Fleet Management System for Autonomous Mobile Robots in Secure Shop-floor Environments,” in Proc. 2021 IEEE 30th Int. Symp. on Industrial Electronics (ISIE), Kyoto, Japan, Jun. 2021, doi: 10.1109/ISIE45552.2021.9576269.

[2] T. Lackner, J. Hermann, C. Kuhn, and D. Palm, “Review of Autonomous Mobile Robots in Intralogistics: State-of-the-Art, Limitations and Research Gaps,” Procedia CIRP, vol. 130, pp. 930-935, 2024.

[3] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, “Fog Computing and Its Role in the Internet of Things,” in Proc. 1st Ed. of the MCC Workshop on Mobile Cloud Computing (ACM SIGCOMM MCC '12), Helsinki, Finland, Aug. 2012, pp. 13-16, doi: 10.1145/2342509.2342513.

[4] P. Mell and T. Grance, “The NIST Definition of Cloud Computing,” NIST Special Publication 800-145, National Institute of Standards and Technology, Gaithersburg, MD, USA, Sep. 2011, doi: 10.6028/NIST.SP.800-145.

[5] K. Krot, G. Iskierka, B. Poskart, and A. Gola, “Predictive Monitoring System for Autonomous Mobile Robots Battery Management Using the Industrial Internet of Things Technology,” Materials, vol. 15, no. 19, p. 6561, Sep. 2022, doi: 10.3390/ma15196561.

[6] Amazon Web Services, “Best Practices for Designing and Using Partition Keys Effectively,” Amazon DynamoDB Developer Guide. [Online]. Available: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-partition-key-design.html

[7] Amazon Web Services, “Increasing Throughput Using Horizontal Scaling and Action Batching with Amazon SQS,” Amazon Simple Queue Service Developer Guide. [Online]. Available: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-throughput-horizontal-scaling-and-batching.html

[8] M. Guerriero, M. Garriga, D. A. Tamburri, and F. Palomba, “Adoption, Support, and Challenges of Infrastructure-as-Code: Insights from Industry,” in Proc. 2019 IEEE Int. Conf. on Software Maintenance and Evolution (ICSME), Cleveland, OH, USA, 2019, pp. 580-589, doi: 10.1109/ICSME.2019.00092.